

Hammers

AI Agents Marketplace — Meeting Copilot

A detailed architecture reference for the Hammers platform: a pnpm monorepo delivering an AI agent that joins Google Meet calls, transcribes them live, analyzes them as they happen, and produces a post-meeting report and follow-up email. This document explains every technical part of the running system with diagrams.

Services	Frontend · Backend · Meeting-Bot fleet · PostgreSQL · Redis
Backend	NestJS 11 · Prisma 6 · PostgreSQL 16
Frontend	React 19 · Vite · Tailwind v4
Bot	Playwright (Chromium) · Deepgram nova-2 · Express
Coordination	Redis — locks · pub/sub · rate-limit

Contents

1. System Overview & Topology
2. Technology Stack
3. Backend Component Map
4. Data Model (Entity Relationships)
5. Meeting Lifecycle State Machine
6. End-to-End Meeting Pipeline (Sequence)
7. Bot Fleet & Registry (Capacity Model)
8. Bot Internals — Audio Capture & Transcription
9. Realtime Transcript Delivery (SSE + Redis Pub/Sub)
10. Redis Responsibilities
11. Incremental Analysis & Reporting
12. API Surface
13. Security & Authentication
14. Deployment Topology
15. Appendix — Environment Variables & Glossary

 Frontend  Backend  Meeting-Bot  Datastore  Redis  External

1 · System Overview & Topology

Hammers is structured as a **pnpm monorepo** with three deployable code units plus two stateful backing services. The **Frontend** is a React SPA. The **Backend** is a NestJS REST API that owns all business logic, persistence, and orchestration. The **Meeting-Bot** is a separate Node + Playwright service — actually a *fleet* of identical containers, each owning one Google account — that joins meetings and streams audio to a speech-to-text provider. **PostgreSQL** is the system of record; **Redis** coordinates the horizontally-scaled backend.

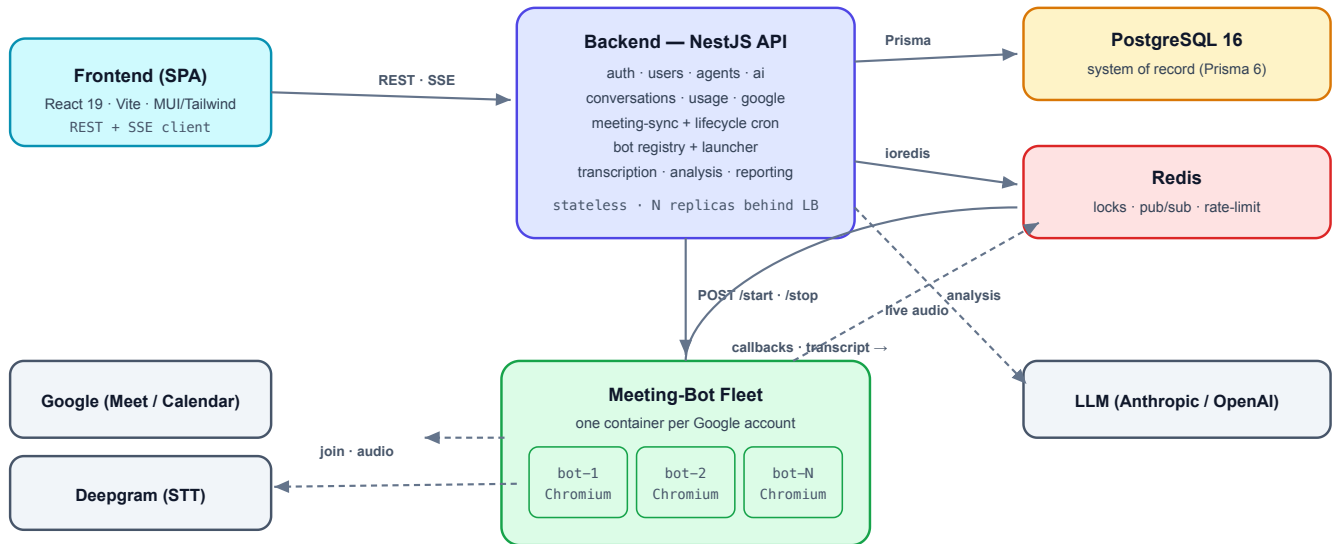


Figure 1 — Runtime topology. Solid = control/data within our system; dashed = external integrations.

The two planes

- **Control plane (Backend)**: decides *when* a bot should join/leave (the lifecycle cron), *which* bot handles a meeting (the registry), persists all state, runs incremental AI analysis, and serves the UI.
- **Data plane (Bot fleet)**: does the heavy, stateful work of one meeting — a headless Chromium session, audio capture, and a live STT websocket — and reports back over HTTP.

2 · Technology Stack

Backend

Concern	Choice	Notes
Framework	NestJS 11 (TypeScript)	Modular DI; one module per bounded context
ORM / DB	Prisma 6 · PostgreSQL 16	Prisma 6 (not 7) for NestJS CJS compatibility
Auth	JWT access + refresh, bcrypt	Passport strategies; refresh-token rotation
Scheduling	@nestjs/schedule	3 jobs: sync, lifecycle, analysis
Coordination	Redis (ioredis)	Distributed locks, pub/sub, throttler store
Rate limiting	@nestjs/throttler	Redis-backed; shared across replicas
Object storage	@azure/storage-blob	Avatars (keeps backend stateless)
AI	Anthropic + OpenAI SDKs	Provider abstraction; SSE streaming
API docs	Swagger (@nestjs/swagger)	/api/docs

Frontend

Concern	Choice
UI	React 19 · Vite
Server state	TanStack React Query 5
Routing	React Router 7
Forms	React Hook Form + Zod
i18n / icons	i18next · lucide-react

Meeting-Bot

Concern	Choice	Notes
Browser automation	Playwright 1.52 (Chromium)	Persistent profile for Google login
Speech-to-text	Deepgram nova-2	Live websocket, diarization, interim results
HTTP control	Express 4	/start · /stop · /health

Infrastructure

Concern	Choice
Monorepo	pnpm workspaces
Local orchestration	docker-compose (postgres · redis · backend · botx2)
Cloud target	Azure (App/Container Apps · PostgreSQL Flexible · Cache for Redis · Blob)

3 · Backend Component Map

Every feature is a NestJS module. Modules depend on a shared **infrastructure** layer (Prisma, Redis) and on each other only through exported services. The `bot` and `meeting-sync` modules form the orchestration core of the Meeting Copilot.

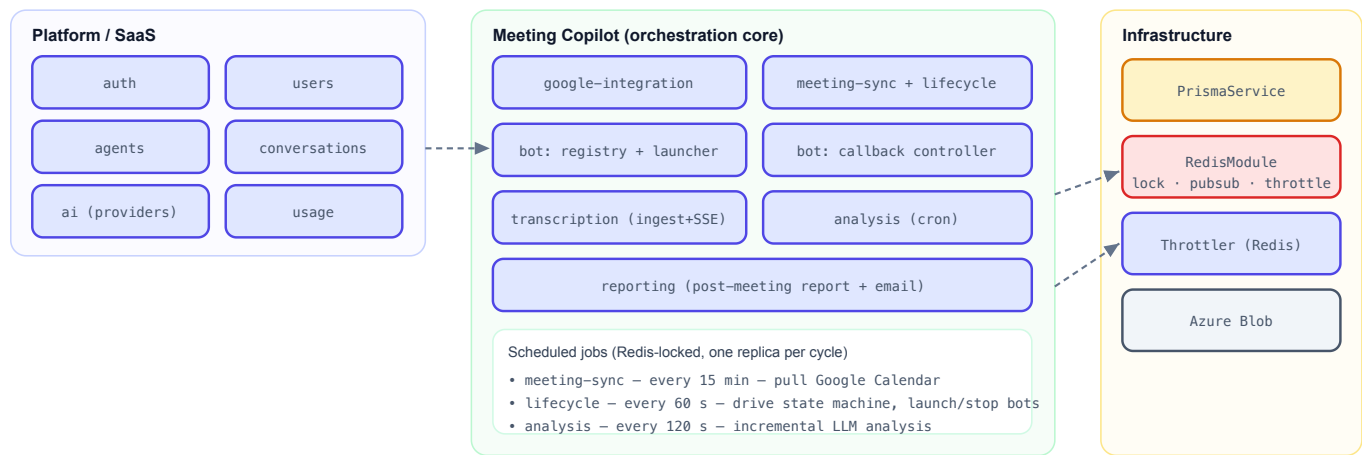


Figure 2 — Module map. The green core is what makes the agent work; the amber column is shared infrastructure used by every module.

Layering rules

- **Controllers** validate input (class-validator DTOs, global `ValidationPipe` with `whitelist`) and delegate to services. No business logic in controllers.
- **Services** hold logic and are the only things that touch Prisma/Redis.
- **Cross-module use** is via an exported service (e.g. `meeting-sync` imports `BotLauncherService` and `ReportingService` from their modules).

4 · Data Model (Entity Relationships)

One `User` connects a Google account and owns many `Meeting` s. Each meeting fans out into one bot job, many transcript segments, and the AI-derived artifacts (analysis, action items, decisions, risks, report, email). A separate cluster handles generic agent chat.

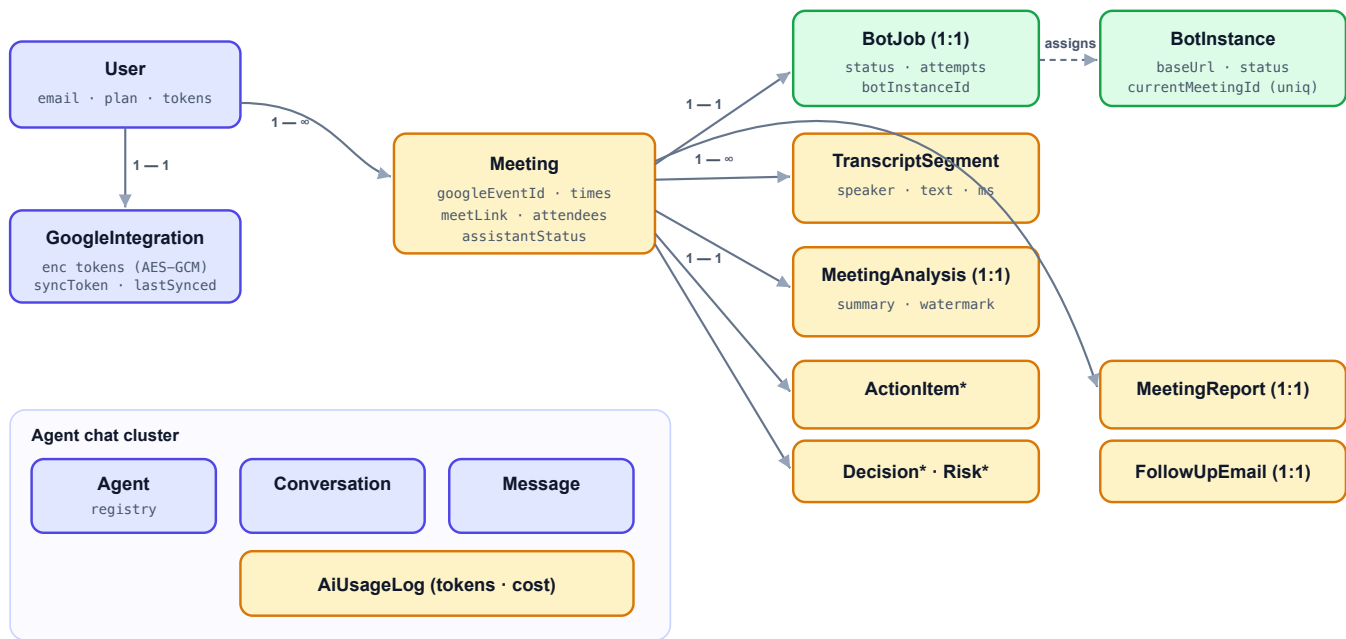


Figure 3 — Core entities. * = many rows per meeting, deduplicated by a normalized `dedupeKey`. All meeting children cascade-delete with the meeting.

Why `BotInstance` is standalone (not a relation of `Meeting`): a bot container's identity and busy/free state live independently of any single meeting. `currentMeetingId` is **unique**, so the database itself guarantees one instance can hold at most one meeting — the backbone of the capacity model in §7.

5 · Meeting Lifecycle State Machine

Every meeting carries an `assistantStatus`. The lifecycle cron (every 60 s, Redis-locked) advances it based on the clock and on bot callbacks. This is the heartbeat of the whole agent.

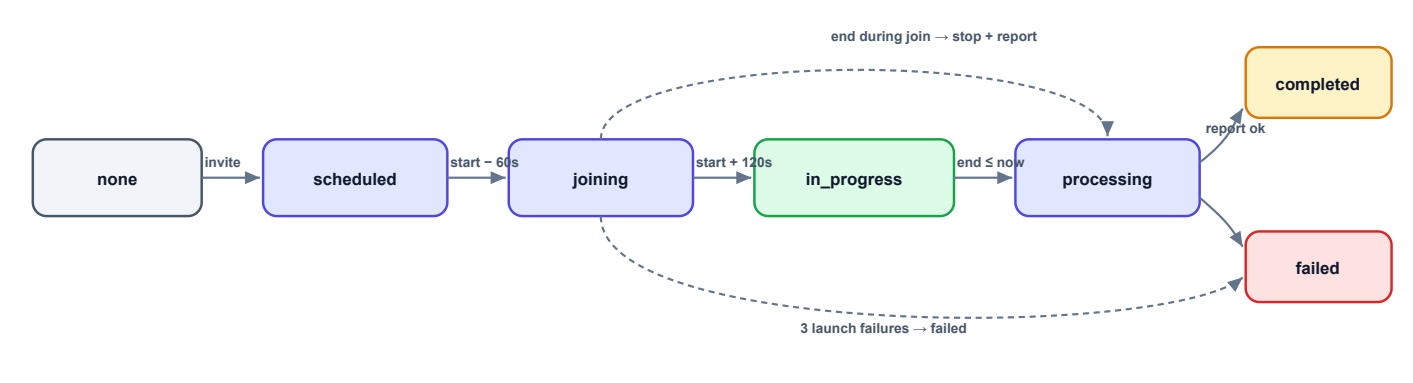


Figure 4 — `assistantStatus` transitions. `JOIN_EARLY = 60s` before start; `JOIN_GRACE = 120s` after start before forcing `in_progress`.

State	Meaning	Entered by
<code>none</code>	Synced from calendar, assistant not requested	meeting-sync
<code>scheduled</code>	User invited the assistant; waiting for start time	<code>POST /meetings/:id/invite</code>
<code>joining</code>	Bot dispatched; logging in / waiting for admission	lifecycle (start - 60s)
<code>in_progress</code>	Bot is in the call, transcribing	lifecycle (start + 120s)
<code>processing</code>	Meeting ended; generating report + email	lifecycle (end ≤ now)
<code>completed</code> / <code>failed</code>	Terminal	reporting / retry exhaustion

6 · End-to-End Meeting Pipeline (Sequence)

The full life of one meeting, from a calendar event to a finished report, across all five services.

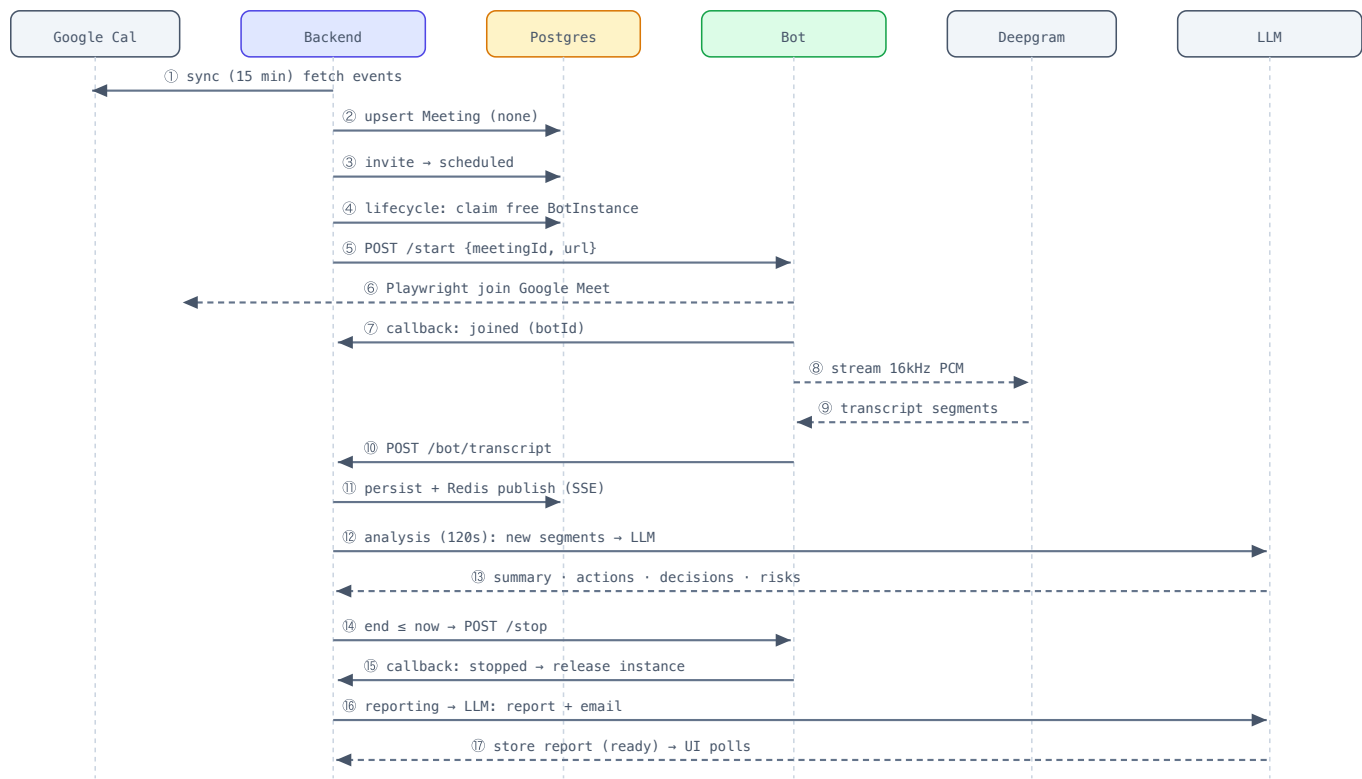


Figure 5 — Solid = our HTTP/DB calls; dashed = external streams/automation. Steps ⑫–⑬ repeat every 2 minutes during the call.

7 · Bot Fleet & Registry (Capacity Model)

Concurrency comes from running **one bot container per Google account**. Each container self-registers with the backend and joins **one meeting at a time**. The backend assigns meetings to free containers, queues when all are busy, and reclaims dead ones — no Google passwords ever leave the bot containers.

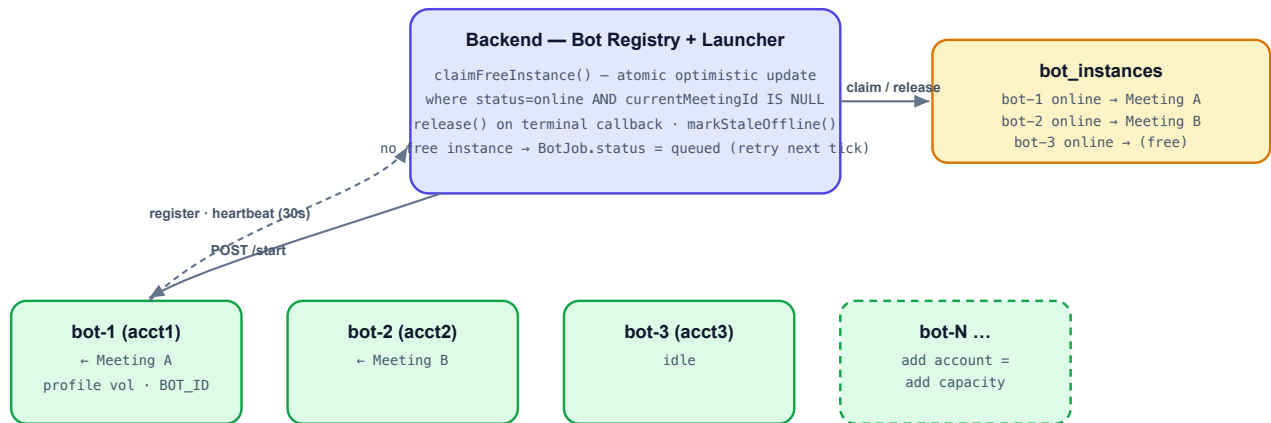


Figure 6 — The registry is the traffic cop. Heartbeats older than 90 s (3 missed) mark an instance offline and free its meeting (reaper).

The claim → queue → release → requeue cycle

- **Claim:** launcher calls `claimFreeInstance`, which atomically flips one free row's `currentMeetingId` (optimistic `updateMany` guarded by `currentMeetingId IS NULL`) — concurrent claims can't grab the same instance.
- **Queue:** if none free, the `BotJob` is marked `queued`; the next lifecycle tick retries it.
- **Release:** a terminal `stopped` / `failed` callback frees the instance.
- **Reap:** a bot that stops heart-beating is marked offline and its meeting released, so a crash never permanently strands a slot.
- **Callback identity:** each callback carries `botId`; the launcher ignores callbacks from an instance other than the one it assigned.

Single-account safety: the bot enforces `MAX_CONCURRENT = 1` and rejects a second `/start` with HTTP 409 `busy` — one Google account is never in two calls at once (a key Google bot-detection risk).

8 · Bot Internals — Audio Capture & Transcription

The hardest engineering is getting meeting audio out of a headless browser. The bot patches WebRTC *inside the page* to tap every remote audio track, mixes and downsamples it, ships it to Node, and streams it to Deepgram.

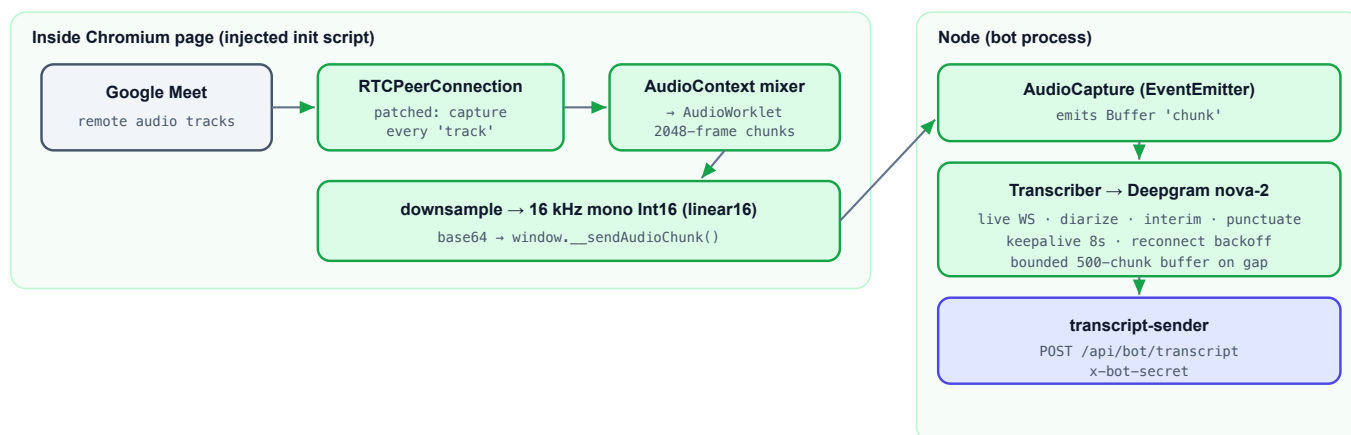


Figure 7 — Audio path. The in-page script runs in the browser; everything right of the boundary runs in Node. Diarization gives per-speaker labels.

Joining the call (Playwright)

- Navigate with `?hl=en` to force English UI; detect hard-fail screens ("you can't join", "not started", "need permission").
- Mute camera & microphone on the pre-join screen via direct DOM clicks (bypasses flaky actionability checks).
- Handle both **"Join now"** (admitted) and **"Ask to join"** (poll up to 5 min for admission).
- Persistent Chrome profile keeps the bot account logged in across runs; `--disable-blink-features=AutomationControlled` + real user-agent reduce bot detection.

9 · Realtime Transcript Delivery (SSE + Redis Pub/Sub)

The browser watches a meeting transcript live via **Server-Sent Events**. Because the backend runs as many replicas, the bot's ingest may land on a different replica than the one holding the user's SSE connection — so delivery is fanned out through **Redis pub/sub**.

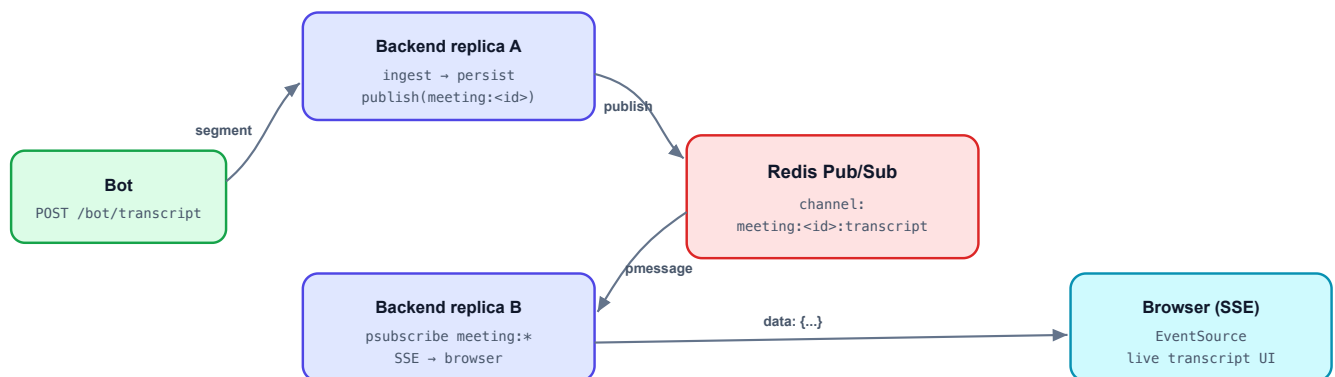


Figure 8 — A segment ingested on replica A reaches a browser connected to replica B via the shared Redis channel. Without Redis, the service degrades to in-process delivery (single-instance dev).

The `TranscriptEventsService` keeps its public API (`publish()` / `stream()`) identical; only the transport changed from an in-memory `RxJS Subject` to Redis pub/sub. Each replica reference-counts local subscribers and unsubscribes when the last viewer disconnects.

10 · Redis Responsibilities

Redis is the coordination substrate that lets the backend run as many identical replicas. It does three distinct jobs:

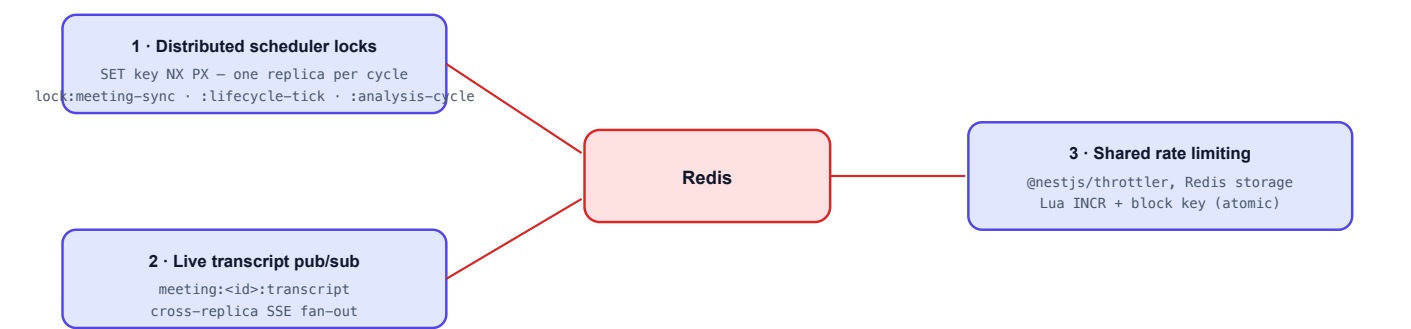


Figure 9 — One Redis, three responsibilities. All three degrade gracefully when `REDIS_URL` is unset (locks run unconditionally, SSE in-process, throttling in-memory).

Job	Mechanism	Problem it solves
Scheduler locks	SET NX PX + Lua release-if-owner	Stops every replica from double-running the same cron (duplicate AI calls, calendar races)
Pub/Sub	psubscribe meeting:*:transcript	Live SSE works regardless of which replica ingest/viewer hit
Rate limiting	Atomic Lua INCR + block key	Auth/AI/bot limits enforced across the whole fleet, not per-process

11 · Incremental Analysis & Reporting

Analysis runs *during* the meeting, not after. A watermark ensures each 2-minute cycle only sends **new** transcript to the model, keeping prompt size and cost proportional to fresh content.

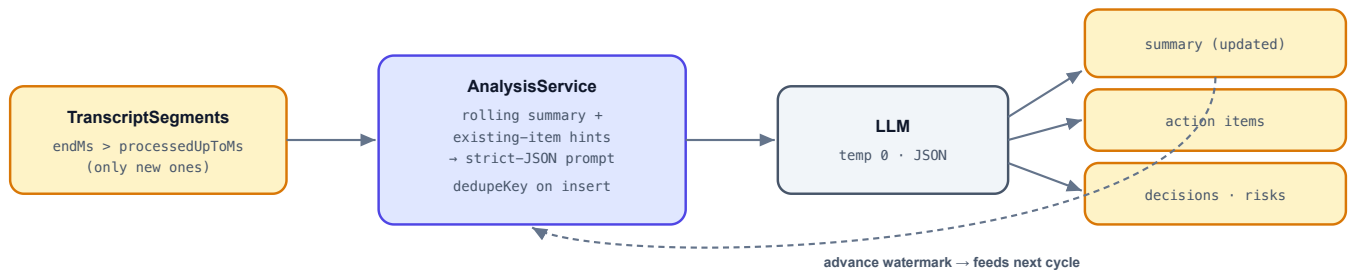


Figure 10 — Incremental analysis loop. Items are deduplicated by a normalized `dedupeKey` (DB unique constraint is the final guard against races).

Post-meeting reporting

- On `processing`, `ReportingService` assembles the rolling summary + extracted items + transcript (capped at 1500 segments) into one prompt.
- The LLM returns a polished **executive summary**, **follow-ups**, and a ready-to-send **email** (subject + body) as strict JSON.
- Status flows `pending` → `generating` → `ready` | `failed`; the UI polls until `ready` and lets the user edit the email before copying.

12 · API Surface

All routes are under the global `/api` prefix. User routes require a JWT; bot routes require the shared `x-bot-secret` header; OAuth callbacks are public.

Group	Routes	Auth
auth	POST <code>/auth/register</code> · <code>/login</code> · <code>/refresh</code> · <code>/logout</code> · <code>/forgot-password</code> · <code>/reset-password</code>	public / refresh
users	GET·PATCH·DELETE <code>/users/me</code> · POST <code>/users/me/avatar</code> · PATCH <code>/users/me/password</code>	JWT
agents	GET·POST <code>/agents</code> · GET·PATCH·DELETE <code>/agents/:id</code>	JWT
ai	POST <code>/ai/chat</code> · POST <code>/ai/chat/stream</code> (SSE)	JWT + throttle
conversations	GET·POST <code>/conversations</code> · GET <code>/conversations/:id[/messages]</code> · POST <code>:id/messages</code> · DELETE <code>:id</code>	JWT
usage	GET <code>/usage/summary</code> · <code>/usage/logs</code>	JWT
google	GET <code>/google/connect</code> · <code>/callback</code> · <code>/status</code> · DELETE <code>/google/disconnect</code>	JWT / public cb
meetings	GET <code>/meetings</code> · <code>/meetings/dashboard</code> · POST <code>/meetings/sync</code> · GET <code>/meetings/sync/status</code> · POST·DELETE <code>/meetings/:id/invite</code>	JWT
transcript	GET <code>/meetings/:id/transcript</code> · GET <code>/meetings/:id/transcript/stream</code> (SSE)	JWT
analysis	GET <code>/meetings/:id/analysis</code> · PATCH <code>/meetings/:id/action-items/:itemId</code>	JWT
reporting	GET <code>/meetings/:id/report</code> · PATCH <code>/meetings/:id/email</code>	JWT
bot (internal)	PATCH <code>/bot/callback</code> · POST <code>/bot/transcript</code> · POST <code>/bot/instances/register</code> · <code>/heartbeat</code>	x-bot-secret + throttle

13 · Security & Authentication

Surface	Mechanism
User auth	JWT access + refresh tokens; bcrypt password hashing; refresh-token hash stored per user; password-reset token with expiry
Bot ↔ backend	Shared <code>x-bot-secret</code> header on all internal endpoints (register, heartbeat, callback, transcript)
Callback integrity	Launcher validates the callback's <code>botId</code> matches the instance assigned to that meeting
Google tokens	Access/refresh tokens encrypted at rest with AES-256-GCM (<code>ENCRYPTION_KEY</code>) before persistence
Rate limiting	Redis-backed throttler on <code>/auth/login</code> , <code>/auth/register</code> , <code>/ai/chat*</code> , and <code>/bot/*</code>
Input validation	Global <code>ValidationPipe</code> (<code>whitelist</code> + <code>forbidNonWhitelisted</code>) on every DTO

Known hardening follow-ups (at higher scale): the bot secret is a single static string (replay risk) — move to per-job signed tokens; add a real `GET /api/health` route (the compose healthcheck currently has none); and runtime-test the Azure Blob avatar path.

14 · Deployment Topology

Locally, `docker-compose` brings up Postgres, Redis, the backend, and a 2-bot fleet. In Azure, each piece maps to a managed service; bot containers scale to zero between meetings so you pay per active meeting-minute.

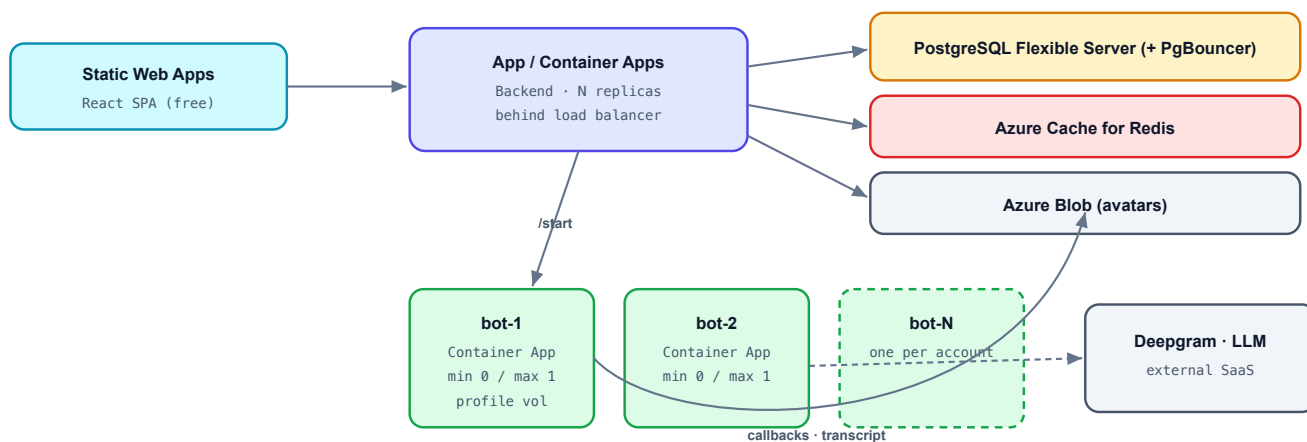


Figure 11 — Azure mapping. Adding meeting capacity = provision a Google account + deploy one more bot Container App (it self-registers).

Component	Local (compose)	Azure
Frontend	vite dev	Static Web Apps (free)
Backend	<code>backend</code> service	App Service / Container Apps (N replicas)
Bot fleet	<code>meeting-bot-1/2</code>	One Container App per account (min 0 / max 1)
Database	<code>postgres</code>	PostgreSQL Flexible Server (+ PgBouncer)
Redis	<code>redis</code>	Azure Cache for Redis
Avatars	Azurite / Blob	Azure Blob Storage

15 · Appendix — Environment & Glossary

Key environment variables

Variable	Service	Purpose
DATABASE_URL	Backend	Postgres DSN (append <code>&connection_limit=N</code>)
REDIS_URL	Backend	Locks · pub/sub · throttle (unset → single-instance mode)
JWT_SECRET / JWT_REFRESH_SECRET	Backend	Token signing
ENCRYPTION_KEY	Backend	AES-256-GCM for Google tokens (64 hex chars)
BOT_CALLBACK_SECRET	Backend + Bot	Shared secret for internal bot endpoints
AZURE_STORAGE_CONNECTION_STRING	Backend	Avatar Blob storage
GOOGLE_CLIENT_ID/SECRET/REDIRECT_URI	Backend	Calendar OAuth
BOT_ID / SELF_URL	Bot	Per-container identity + self-registration URL
BOT_GOOGLE_EMAIL/PASSWORD	Bot	The single Google account this container owns
DEEPGRAM_API_KEY	Bot	Live speech-to-text

Glossary

Term	Meaning
BotInstance	A registered bot container (one Google account) tracked by the backend
Claim / Release	Atomically assigning / freeing a BotInstance for a meeting
Reaper	Lifecycle logic that frees instances with stale heartbeats or ended meetings
Watermark	<code>processedUpToMs</code> — how far analysis has consumed the transcript
dedupeKey	Normalized text key preventing duplicate action items / decisions / risks
Diarization	Deepgram labeling which speaker said each segment
SSE	Server-Sent Events — one-way live stream from server to browser

Document scope: this describes the system after the multi-bot-fleet & multi-user scaling work (registry, Redis coordination, stateless backend). For scaling to a few hundred concurrent meetings, the orchestration would move to a queue-driven autoscaling pool and the Google-account supply / STT-LLM cost become the dominant constraints.